1. Introduction

1.1 Visualization Library

This tutorial demonstrates the use of **Altair**, a declarative statistical visualization library built on top of Vega-Lite. It was developed by JakeVanderPlas and Brian Granger. Altair is designed for concise, high-quality, and interactive data visualization.

Why Use Altair?

Altair provides an intuitive and efficient way to generate complex visualizations with minimal code. Unlike Matplotlib and Seaborn, Altair follows a declarative approach, allowing users to specify what they want to visualize rather than focusing on how it should be drawn.

Advantages of Altair

- **Declarative Syntax:** Altair's declarative nature means users define relationships between data and encodings (e.g., axes, colors, tooltips) rather than manually setting up every detail.
- **Built-in Interactivity:** Unlike Matplotlib, which requires additional coding for interactivity, Altair provides built-in zooming, filtering, selection, and linked charts with just a few lines of code.
- **Automatic Best Practices:** Altair automatically optimizes visualizations by handling axis scaling, legends, and data binning without requiring extra configuration.
- **Seamless Integration with Pandas:** Since Altair works natively with Pandas DataFrames, it allows for smooth data manipulation and visualization.
- **Perfect for Dashboards & Storytelling:** Altair's strength lies in its interactive capabilities, making it a great choice for data exploration dashboards.

Comparison: Altair vs. Other Visualization Libraries

Feature	Altair	Matplotlib	Seaborn	Plotly
Code Complexity	Low (declarative)	High (procedural)	Medium	Medium
Interactivity	Built-in	None (static)	None (static)	High
Customization	Moderate	High	Medium	High
Best Use Case	Dashboards, Exploratory Analysis	Static Reports	Statistical Charts	Web Apps

Limitations of Altair

❌ **Not Suitable for Large Datasets** – Vega-Lite enforces a row limit of 5000 rows in Jupyter Notebooks. This can be bypassed by aggregating data before visualization.
❌ **Limited Customization** – Altair enforces best practices by default, meaning users have less control over minor design elements compared to Matplotlib.
❌ **No 3D Visualization Support** – Unlike Plotly, Altair does not support 3D plots, making it unsuitable for volumetric data.

Installation Instructions

To use Altair in Jupyter Notebook, install it using:

```
pip install altair
```

2. Import Libraries

The libraries listed below will be utilized for data cleaning and visualizations.

In [3]:
```
import altair as alt

import numpy as np
import pandas as pd
from scipy.stats import linregress
```

3. Dataset Cleaning and Preprocessing

3.1 Overview

The dataset used in this analysis comes from the **EPA Automotive Trends Report**, which contains historical data on fuel economy (MPG), CO_2 emissions, and vehicle attributes. We focus on model years 2010 onward to analyze modern automotive trends.

- **Link**: https://www.epa.gov/automotive-trends/explore-automotive-trends-data

3.2 Dataset Details

This project uses the following dataset:

- **manufacturer_epa.csv** - Provides fuel economy, CO_2 emissions, and performance metrics at the manufacturer level.

3.3 Data Cleaning Overview

Before building visualizations, we must prepare the dataset:

- Convert numerical columns from object type to float (e.g., Model Year, MPG, CO_2 emissions).
- Filter dataset to include only model years from 2010 onward.
- Check for missing values and handle them appropriately.
- Ensure consistency in column names and types.

4. Data Cleaning Steps

Load the Manufacturer Dataset.

In [7]:
```
manufacturer_epa_path = "manufacturer_epa.csv"
manufacturer_epa_df = pd.read_csv(manufacturer_epa_path)
```

Display the first few rows of the Manufacturer Dataset.

In [9]:
```
manufacturer_epa_df.head()
```

Out[9]:

	Manufacturer	Model Year	Regulatory Class	Real-World MPG	Real-World MPG_City	Real-World MPG_Hwy	Real-World CO2 (g/mi)	Real-World CO2_City (g/mi)	Real-World CO2_Hwy (g/mi)	Weight (lbs)	Horsepower (HP)	Footprint (sq. ft.)
0	All	1975	All	13.05970	12.01552	14.61167	680.59612	739.73800	608.31160	4060.399	137.3346	-
1	All	1976	All	14.22136	13.18117	15.73946	625.02238	674.34147	564.74348	4079.198	135.0839	-
2	All	1977	All	15.06743	14.00580	16.60587	589.99880	634.71366	535.34732	3981.818	135.9847	-
3	All	1978	All	15.83777	14.68193	17.52390	561.62442	605.82637	507.59981	3715.238	129.0248	-
4	All	1979	All	15.91271	14.87711	17.39245	559.69495	598.63764	512.09833	3655.465	123.5922	-

Let's check the data types for the manufacturer dataset.

In [11]:
```
print("Manufacturer Dataset Data Types:\n", manufacturer_epa_df.dtypes)
```

```
Manufacturer Dataset Data Types:
 Manufacturer                  object
Model Year                    object
Regulatory Class              object
Real-World MPG                object
Real-World MPG_City           object
Real-World MPG_Hwy            object
Real-World CO2 (g/mi)         object
Real-World CO2_City (g/mi)    object
Real-World CO2_Hwy (g/mi)     object
Weight (lbs)                  object
Horsepower (HP)               object
Footprint (sq. ft.)           object
dtype: object
```

We need to convert numerical columns from object to float for analysis. We also need to cover the Model Year column to an int for analysis.

In [13]:
```
num_columns = [
    "Model Year", "Real-World MPG", "Real-World MPG_City", "Real-World MPG_Hwy",
    "Real-World CO2 (g/mi)", "Real-World CO2_City (g/mi)", "Real-World CO2_Hwy (g/mi)",
    "Weight (lbs)", "Horsepower (HP)", "Footprint (sq. ft.)"
]

manufacturer_epa_df[num_columns] = manufacturer_epa_df[num_columns].apply(pd.to_numeric, errors="coerce")
manufacturer_epa_df["Model Year"] = manufacturer_epa_df["Model Year"].astype("Int64")

print(manufacturer_epa_df.dtypes)
```

```
Manufacturer                  object
Model Year                     Int64
Regulatory Class              object
Real-World MPG               float64
Real-World MPG_City          float64
Real-World MPG_Hwy           float64
Real-World CO2 (g/mi)        float64
Real-World CO2_City (g/mi)   float64
Real-World CO2_Hwy (g/mi)    float64
Weight (lbs)                 float64
Horsepower (HP)              float64
Footprint (sq. ft.)          float64
dtype: object
```

Now, we need to filter the dataset for model years 2010 and later to analyze modern automotive trends related to fuel economy (MPG), CO_2 emissions, and vehicle attributes.

In [15]:
```
manufacturer_epa_df = manufacturer_epa_df[manufacturer_epa_df["Model Year"] >= 2010]

manufacturer_epa_df.head()
```

Out[15]:

	Manufacturer	Model Year	Regulatory Class	Real-World MPG	Real-World MPG_City	Real-World MPG_Hwy	Real-World CO2 (g/mi)	Real-World CO2_City (g/mi)	Real-World CO2_Hwy (g/mi)	Weight (lbs)	Horsepower (HP)	Footprint (sq. ft.)
35	All	2010	All	22.59206	19.11219	26.18930	393.65429	465.33221	339.58148	4001.323	213.6361	48.54913
36	All	2011	All	22.28844	18.83713	25.86317	398.99558	472.11781	343.83319	4125.934	229.9718	49.54439
37	All	2012	All	23.56593	19.94669	27.30319	377.31888	445.79746	325.65960	3978.812	221.7796	48.81134
38	All	2013	All	24.17888	20.49116	27.97717	367.53789	433.74031	317.59572	4002.973	225.8506	49.08053
39	All	2014	All	24.11047	20.44020	27.88816	368.65513	434.90361	318.67820	4059.639	230.2484	49.72043

If we look at the Dataset, we see these unique Car Manufacturers:

In [17]:
```
car_manufacturers = manufacturer_epa_df["Manufacturer"].unique()

print(car_manufacturers)
```

```
['All' 'BMW' 'Ford' 'GM' 'Honda' 'Hyundai' 'Kia' 'Mazda' 'Mercedes'
 'Nissan' 'Stellantis' 'Subaru' 'Tesla' 'Toyota' 'VW']
```

We want to filter out the All Car Manufacturers Option, since we don't want to be doing analysis on aggregate data.

In [19]:
```
manufacturer_epa_df = manufacturer_epa_df[manufacturer_epa_df["Manufacturer"] != "All"]

manufacturer_epa_df.head()
```

Out[19]:

	Manufacturer	Model Year	Regulatory Class	Real-World MPG	Real-World MPG_City	Real-World MPG_Hwy	Real-World CO2 (g/mi)	Real-World CO2_City (g/mi)	Real-World CO2_Hwy (g/mi)	Weight (lbs)	Horsepower (HP)	Footprint (sq. ft.)
85	BMW	2010	All	22.11894	18.26597	26.30477	403.87825	489.03931	339.63393	3899.421	254.9807	45.78178
86	BMW	2011	All	22.64959	18.71925	26.91229	394.58177	477.44425	332.07147	4045.147	262.2890	46.78290
87	BMW	2012	All	23.53401	19.74701	27.51463	380.20532	453.05973	325.24498	4070.738	264.2034	47.33868
88	BMW	2013	All	24.30114	20.21850	28.66816	366.24242	440.17913	310.46561	4012.190	267.1041	47.37450
89	BMW	2014	All	26.10456	21.73755	30.76751	340.99678	409.76701	289.11748	4016.625	264.8449	47.83606

5. Creating a Correlation Heatmap

We will create a correlation table and a heatmap to identify which fields are best for generating charts.

In [21]:
```
manufacturer_epa_df_corr = manufacturer_epa_df.drop(columns=['Manufacturer', 'Model Year', 'Regulatory Class']).corr()
manufacturer_epa_df_corr
```

Out[21]:

	Real-World MPG	Real-World MPG_City	Real-World MPG_Hwy	Real-World CO2 (g/mi)	Real-World CO2_City (g/mi)	Real-World CO2_Hwy (g/mi)	Weight (lbs)	Horsepower (HP)	Footprint (sq. ft.)
Real-World MPG	1.000000	0.999087	0.999248	-0.942325	-0.929937	-0.951147	0.224892	0.624383	0.142867
Real-World MPG_City	0.999087	1.000000	0.996743	-0.931691	-0.919647	-0.940200	0.233201	0.631641	0.151653
Real-World MPG_Hwy	0.999248	0.996743	1.000000	-0.949318	-0.936128	-0.958948	0.214880	0.616615	0.132379
Real-World CO2 (g/mi)	-0.942325	-0.931691	-0.949318	1.000000	0.997956	0.997771	-0.037447	-0.437486	0.020574
Real-World CO2_City (g/mi)	-0.929937	-0.919647	-0.936128	0.997956	1.000000	0.991467	-0.023020	-0.414921	0.029797
Real-World CO2_Hwy (g/mi)	-0.951147	-0.940200	-0.958948	0.997771	0.991467	1.000000	-0.052340	-0.459132	0.010860
Weight (lbs)	0.224892	0.233201	0.214880	-0.037447	-0.023020	-0.052340	1.000000	0.846337	0.858628
Horsepower (HP)	0.624383	0.631641	0.616615	-0.437486	-0.414921	-0.459132	0.846337	1.000000	0.696805
Footprint (sq. ft.)	0.142867	0.151653	0.132379	0.020574	0.029797	0.010860	0.858628	0.696805	1.000000

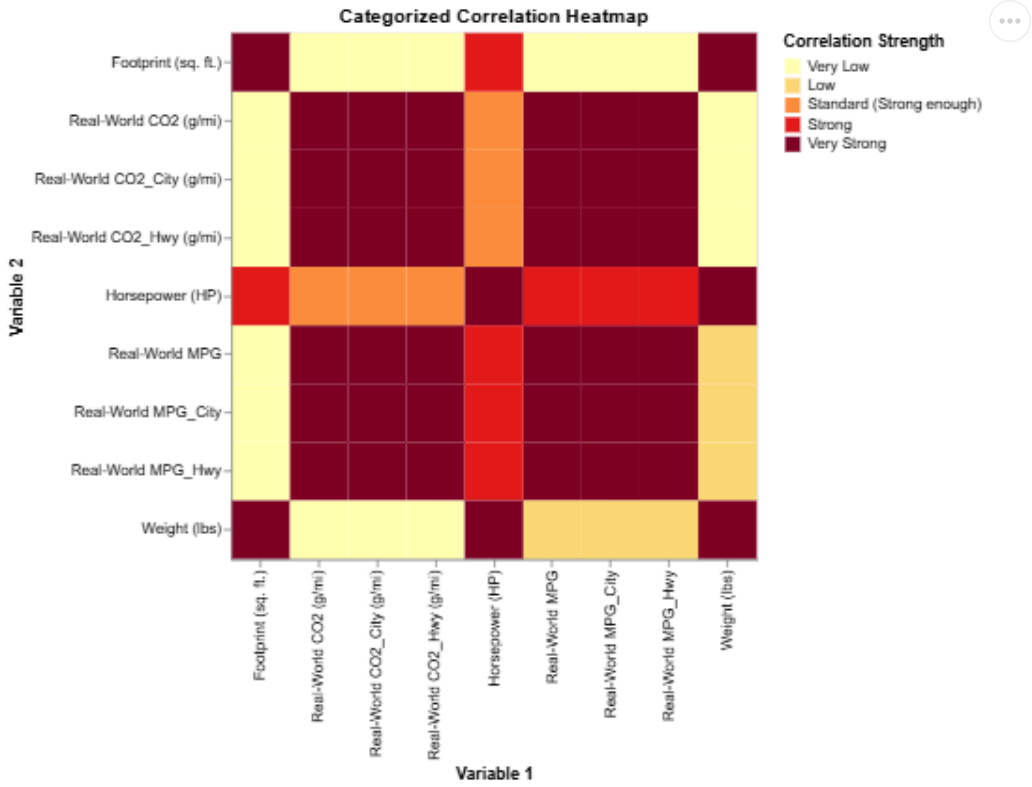
Now that we have created a correlation table, our next step is to create a heatmap. We also want to categorize the correlations, ranging from Very Low to Very Strong. This will help us analyze the data more effectively.

This line suppresses FutureWarning messages in Python, preventing them from being displayed. This ensures that the dashboard doesn't become cluttered.

In [24]:
```
import warnings

warnings.simplefilter(action='ignore', category=FutureWarning)
```

In [25]:
```
def categorize_correlation(value):
    if abs(value) < 0.2:
        return 'Very Low'
    elif 0.2 <= abs(value) < 0.4:
        return 'Low'
    elif 0.4 <= abs(value) < 0.6:
        return 'Standard (Strong enough)'
    elif 0.6 <= abs(value) < 0.8:
        return 'Strong'
    else:
        return 'Very Strong'

correlation_categories = manufacturer_epa_df_corr.map(categorize_correlation)

corr_long = correlation_categories.reset_index().melt(id_vars='index')
corr_long.columns = ['Variable 1', 'Variable 2', 'Correlation Category']

color_scale = alt.Scale(
    domain=['Very Low', 'Low', 'Standard (Strong enough)', 'Strong', 'Very Strong'],
    range=['#ffffb2', '#fed976', '#fd8d3c', '#e31a1c', '#800026']
)

heatmap = alt.Chart(corr_long).mark_rect().encode(
    x=alt.X(
        'Variable 1:N',
        title='Variable 1',
        sort=alt.EncodingSortField(field="Correlation Category", order='descending')
    ),
    y=alt.Y(
        'Variable 2:N',
        title='Variable 2',
        sort=alt.EncodingSortField(field="Correlation Category", order='descending')
    ),
    color=alt.Color(
        'Correlation Category:N',
        scale=color_scale,
        legend=alt.Legend(title="Correlation Strength")
    ),
    tooltip=['Variable 1', 'Variable 2', 'Correlation Category']
)

heatmap = heatmap.properties(title="Categorized Correlation Heatmap", width=350, height=350)

heatmap
```

Out[25]:



5.1 Key Insights from the Correlation Table

- **Fuel Economy vs. CO_2 Emissions**: There is a strong negative correlation (-0.94 to -0.96) between MPG and CO_2 emissions, meaning that vehicles with higher fuel economy produce significantly lower CO_2 emissions.
- **Horsepower vs. Weight**: A strong positive correlation (0.84) exists between horsepower and vehicle weight, confirming that heavier vehicles typically have more powerful engines.
- **MPG vs. Horsepower**: A moderate positive correlation (0.62) suggests that higher-horsepower vehicles can still achieve good fuel economy, likely due to advancements in engine efficiency and hybrid technology.
- **Weight vs. Fuel Economy**: A weak positive correlation (0.22) suggests that vehicle weight does not strongly determine MPG, possibly due to efficiency optimizations in modern vehicles.

6. Plotting the Data

Now, it is time to create the Visualizations.

Visualization #1: Trend of Real-World Average MPG Over Model Years

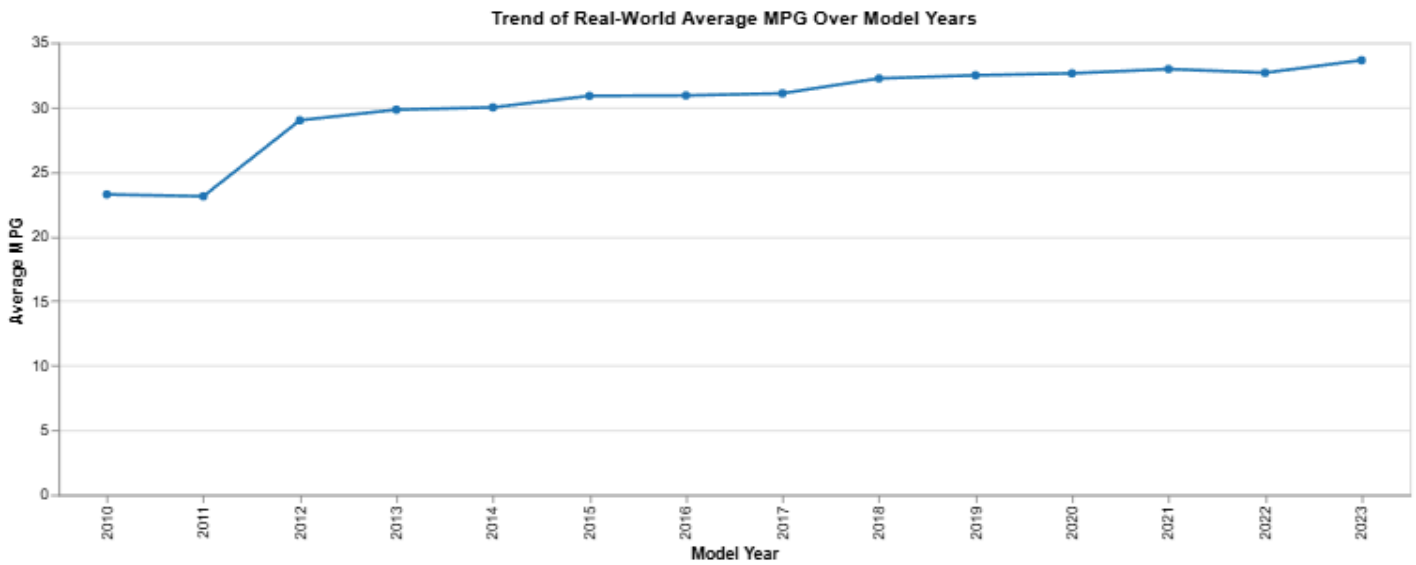
- This line chart illustrates the trend of Real-World Average MPG for vehicles across different model years.
- Each point represents the Average MPG for a given model year, with a tooltip providing exact values upon hovering.
- The blue line visually tracks the progression, highlighting improvements in fuel efficiency over time.

Step 1: Creating a Basic Line Chart

In this step, we create a simple line chart that shows the trend of Average MPG over different model years. This chart does not yet include tootltips.

In [30]:
```
line_chart = alt.Chart(manufacturer_epa_df).mark_line(point=True).encode(
    x=alt.X('Model Year:O', title="Model Year"),
    y=alt.Y('mean(Real-World MPG):Q', title="Average MPG"),
    color=alt.value("#1f77b4")
)

line_chart = line_chart.properties(title="Trend of Real-World Average MPG Over Model Years", height=300, width=900)

line_chart
```

Out[30]:



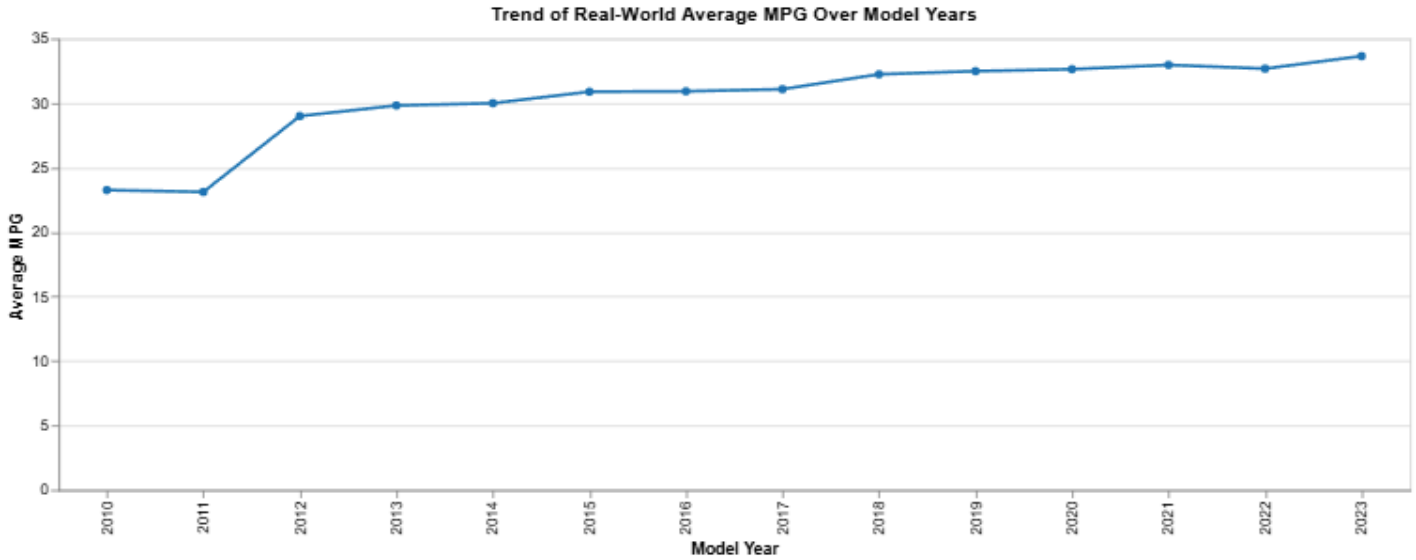
Step 2: Enhancing with Interactivity

Now, we enhance the line chart by adding tooltips, which allow users to hover over points to see the exact values for Model Year and Average MPG. This makes the line chart more informative and interactive.

In [32]:
```
line_chart = alt.Chart(manufacturer_epa_df).mark_line(point=True).encode(
    x=alt.X('Model Year:O', title="Model Year"),
    y=alt.Y('mean(Real-World MPG):Q', title="Average MPG"),
    color=alt.value("#1f77b4"),
    tooltip=[
    alt.Tooltip('Model Year:O', title="Model Year"),
    alt.Tooltip('mean(Real-World MPG):Q', title="Average MPG", format=".3f")
```

```
    ]
)

line_chart = line_chart.properties(title="Trend of Real-World Average MPG Over Model Years", height = 300, width = 900)

line_chart
```

Out[32]:



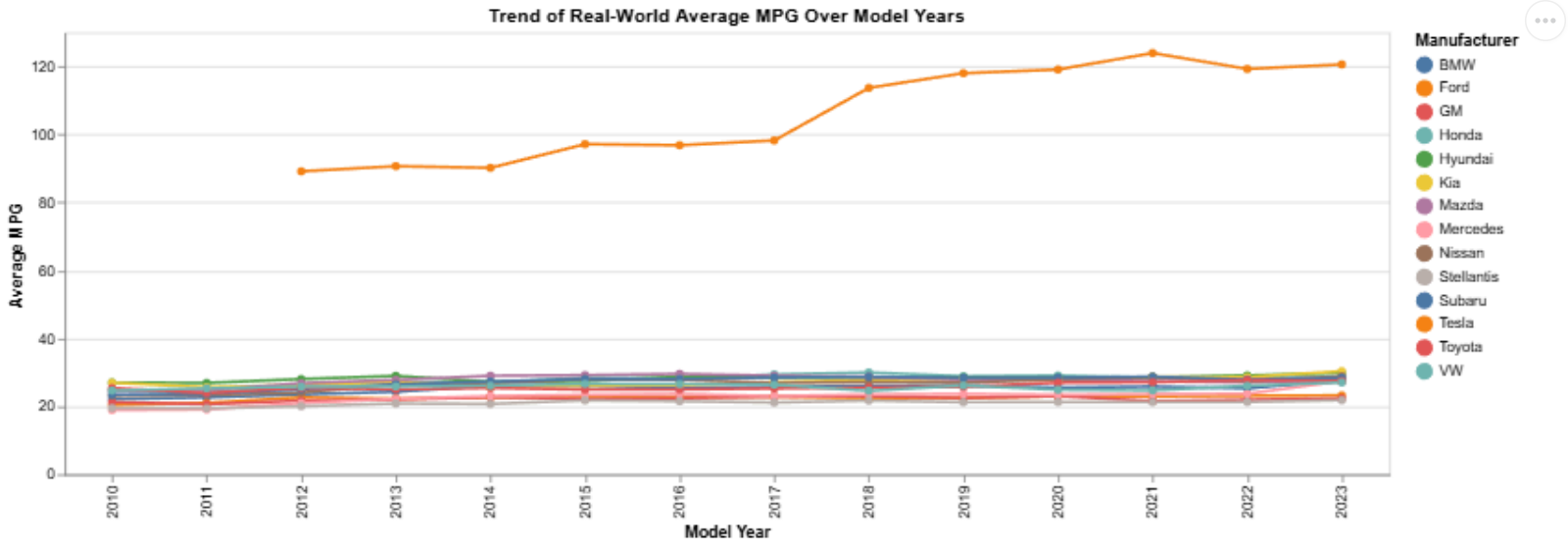
Step 3: Adding Color Encoding

- Now, we add a color encoding to the line chart, which assigns a unique color (or hue) for each Manufacturer.
- This allows us to easily compare the Average MPG across different manufacturers over the specified model years.

In [34]:

```
line_chart = alt.Chart(manufacturer_epa_df).mark_line(point=True).encode(
    x=alt.X('Model Year:O', title="Model Year"),
    y=alt.Y('mean(Real-World MPG):Q', title="Average MPG"),
    color=alt.Color('Manufacturer:N', title="Manufacturer"),
    tooltip=[
        alt.Tooltip('Manufacturer:N', title="Manufacturer"),
        alt.Tooltip('Model Year:O', title="Model Year"),
        alt.Tooltip('Real-World MPG:Q', title="Real-World MPG", format=".3f")
    ]
)

line_chart = line_chart.properties(
    title="Trend of Real-World Average MPG Over Model Years",
    height=300,
    width=900
)

line_chart
```

Out[34]:



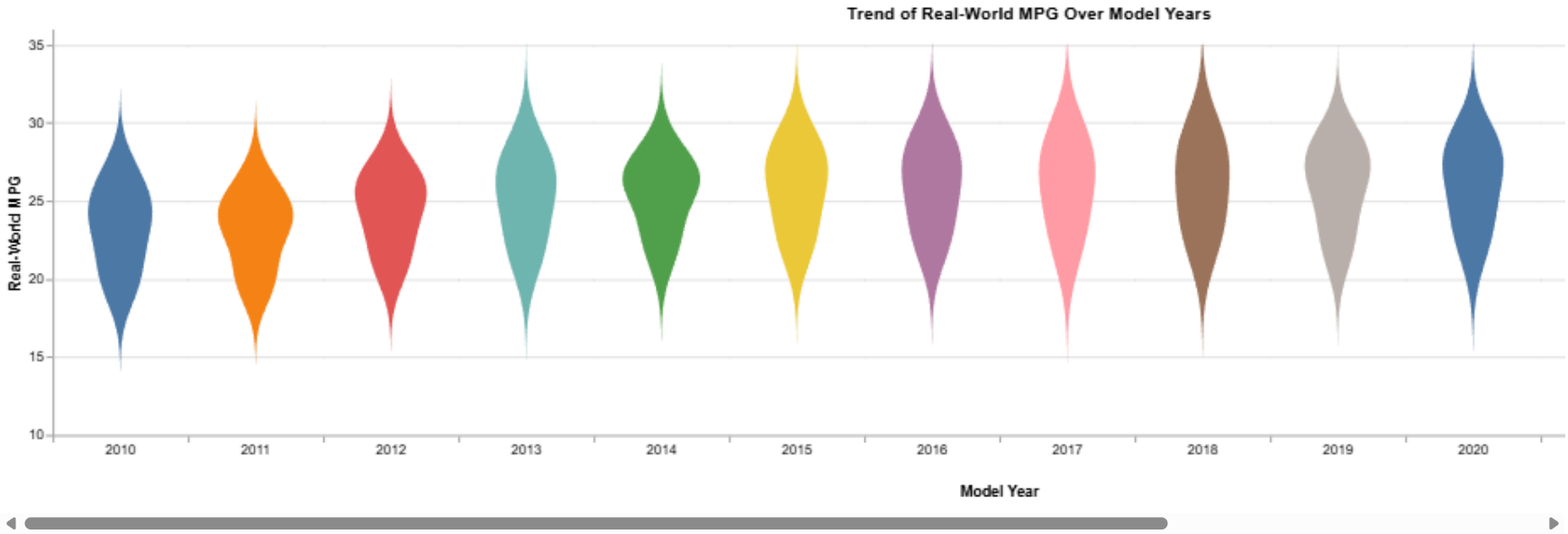
Step 4: Creating the Violin Plot

We can also create a Violing Plot to visualize the distribution of Real-World MPG over different Model Years.

In [36]:

```
violin_plot = alt.Chart(manufacturer_epa_df).transform_density(
    'Real-World MPG',
    as_=['Real-World MPG', 'density'],
    extent = [10, 35],
    groupby=['Model Year']
).mark_area(orient='horizontal').encode(
    alt.X('density:Q', stack='center', impute=None, title=None, axis=alt.Axis(labels=False, values=[0], grid=False, ticks=True)),
    alt.Y('Real-World MPG:Q', title="Real-World MPG"),
    alt.Color('Model Year:N', title="Model Year"),
    alt.Column('Model Year:N', spacing=0, header=alt.Header(titleOrient='bottom', labelOrient='bottom', labelPadding=0)),
    tooltip=[
        alt.Tooltip('Model Year:O', title="Model Year"),
        alt.Tooltip('density:Q', title="Density"),
        alt.Tooltip('Real-World MPG:Q', title="Real-World MPG", format=".3f")
    ]
)

violin_plot = violin_plot.properties(
    title=alt.TitleParams(
        text="Trend of Real-World MPG Over Model Years",
        anchor="middle"
    ),
    width = 100
)

violin_plot = violin_plot.configure_view(stroke=None)

violin_plot
```

Out[36]:



Conclusion:

- The trend of real-world MPG has increased over the years, partially due to advancements in fuel efficiency technologies and the introduction of electric vehicles (EVs).

Visualization #2: Relationship Between MPG and Horsepower (HP)

- This scatter plot explores the relationship between Real-World MPG and Horsepower (HP).
- Each point represents a vehicle, color-coded by manufacturer.
- The tooltip provides details such as car brand, model year, weight, and MPG for better insights.

Step 1: Creating the Initial Scatter Plot

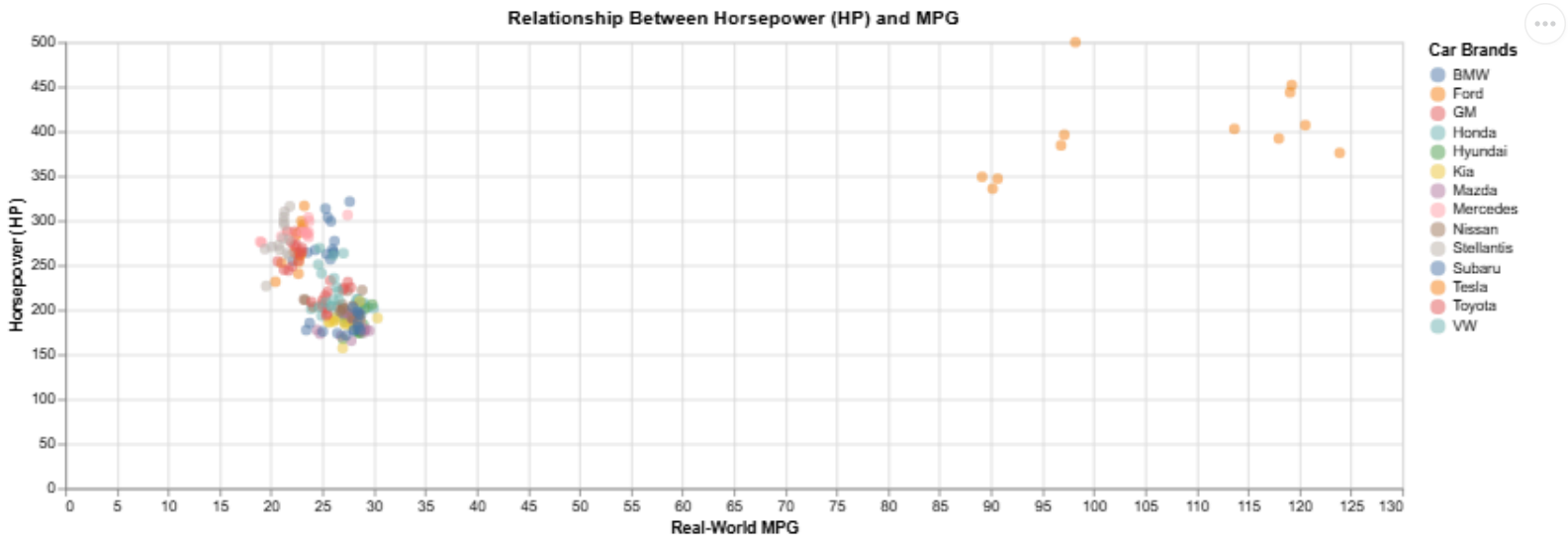
In this step, we create a scatter plot that visualizes how horsepower affects fuel economy (MPG). Each point represents a different vehicle, categorized by manufacturer.

In [40]:

```
scatter_plot = alt.Chart(manufacturer_epa_df).mark_circle(size=50, opacity=0.5).encode(
    x=alt.X('Real-World MPG:Q', title="Real-World MPG"),
    y=alt.Y(
        'Horsepower (HP):Q', title="Horsepower (HP)"
    ),
    color=alt.Color('Manufacturer:N', title="Car Brands", legend=alt.Legend(title="Car Brands")),
    tooltip=[
        alt.Tooltip('Manufacturer:N', title="Car Brand"),
        alt.Tooltip('Model Year:O', title="Model Year"),
        alt.Tooltip('Weight (lbs):Q', title="Vehicle Weight (lbs)", format=".3f"),
        alt.Tooltip('Real-World MPG:Q', title="Real-World MPG", format=".3f")
    ]
)
```

```
scatter_plot = scatter_plot.properties(title="Relationship Between Horsepower (HP) and MPG", height=300, width=900)

scatter_plot
```

Out[40]:



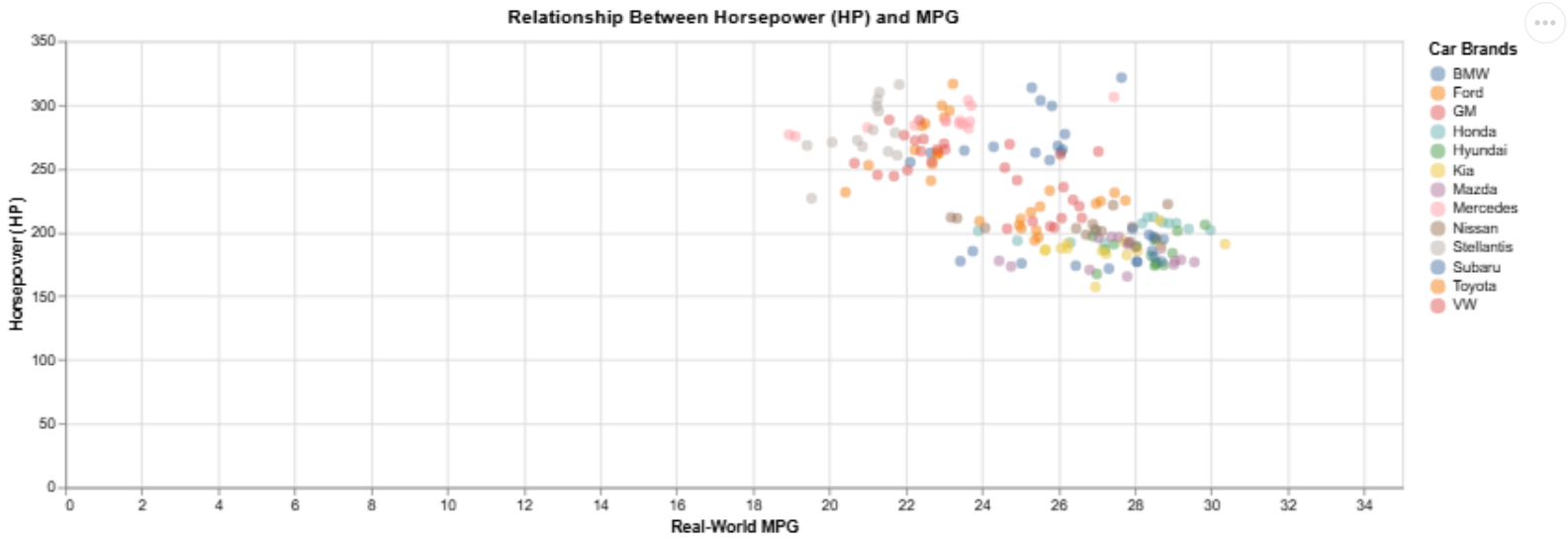
Step 2: Filtering Out Tesla Vehicles

We remove Tesla from the dataset to analyze fuel economy without electric vehicles. This helps focus on traditional internal combustion engine (ICE) vehicles since EVs are naturally heavier due to their battery packs and get better gas mileage, too.

In [42]:

```
manufacturer_epa_df = manufacturer_epa_df[manufacturer_epa_df["Manufacturer"] != "Tesla"]

scatter_plot = alt.Chart(manufacturer_epa_df).mark_circle(size=50, opacity=0.5).encode(
    x=alt.X('Real-World MPG:Q', title="Real-World MPG"),
    y=alt.Y(
        'Horsepower (HP):Q', title="Horsepower (HP)"
    ),
    color=alt.Color('Manufacturer:N', title="Car Brands", legend=alt.Legend(title="Car Brands")),
    tooltip=[
        alt.Tooltip('Manufacturer:N', title="Car Brand"),
        alt.Tooltip('Model Year:O', title="Model Year"),
        alt.Tooltip('Weight (lbs):Q', title="Vehicle Weight (lbs)", format=".3f"),
        alt.Tooltip('Real-World MPG:Q', title="Real-World MPG", format=".3f")
    ]
)

scatter_plot = scatter_plot.properties(title="Relationship Between Horsepower (HP) and MPG", height=300, width=900)

scatter_plot
```

Out[42]:



Conclusion:

- Initially, the scatter plot suggests a moderate positive correlation (0.62) between horsepower and fuel economy (MPG), likely due to the presence of hybrid and electric vehicles, which combine high horsepower with efficiency.
- However, after removing Tesla vehicles, this correlation weakens significantly, indicating that traditional internal combustion engine (ICE) vehicles still exhibit the expected trend of higher horsepower leading to lower MPG.
- This suggests that advancements in hybrid and electric powertrains play a crucial role in maintaining fuel efficiency despite increased horsepower.

Visualization #3: Fuel Economy vs. CO_2 Emissions

- This scatter plot examines the relationship between fuel economy (MPG) and CO_2 emissions (g/mi).
- There is a strong negative correlation (-0.94 to -0.96), meaning that vehicles with higher MPG produce significantly lower CO_2 emissions.
- Additional interactivity, such as brand filtering, tooltips, and zooming enhances the visualization.

Step 1: Filtering Out Tesla Vehicles

We remove Tesla from the dataset to analyze the data without electric vehicles. This helps focus on traditional internal combustion engine (ICE) vehicles since EVs don't have CO_2 emissions (g/mi).

In [46]:
```
manufacturer_epa_df = manufacturer_epa_df[manufacturer_epa_df["Manufacturer"] != "Tesla"]
```

Step 2: Adding a Dropdown for Brand Selection

To allow users to filter vehicles by manufacturer, we create a dropdown selection.
This enables users to view data for a specific car brand or choose `"All Brands"` to see the complete dataset.

In [48]:
```
# Define the dropdown selection for filtering by manufacturer
car_manufacturers = ["All Brands"] + manufacturer_epa_df["Manufacturer"].unique().tolist()

# Create a dropdown menu to select car brands
dropdown = alt.binding_select(options=car_manufacturers, name="Select Brand: ")
selection = alt.param(name="Manufacturer", bind=dropdown, value="All Brands")
```

Step 3: Building the Scatter Plot with Tooltips

- Each point represents a vehicle, color-coded by manufacturer.
- Enhanced tooltips display details like horsepower, weight, and footprint when hovering over a point.
- The chart scales Real-World MPG between 18-32 to focus on typical vehicle ranges.

In [50]:
```
scatter = alt.Chart(manufacturer_epa_df).mark_circle(size=60, opacity=0.5).encode(
    x=alt.X('Real-World MPG:Q', title="Real-World MPG", scale=alt.Scale(domain=[18, 32])),
    y=alt.Y('Real-World CO2 (g/mi):Q', title="CO2 Emissions (g/mi)"),
    color=alt.Color('Manufacturer:N', title="Car Brands", legend=alt.Legend(title="Car Brands")),
    tooltip=[
        alt.Tooltip('Manufacturer:N', title="Car Brand"),
        alt.Tooltip('Real-World MPG:Q', title="Real-World MPG", format=".3f"),
        alt.Tooltip('Real-World CO2 (g/mi):Q', title="CO2 Emissions (g/mi)", format=".3f"),
        alt.Tooltip('Weight (lbs):Q', title="Weight (lbs)", format=".3f"),
        alt.Tooltip('Horsepower (HP):Q', title="Horsepower (HP)", format=".3f"),
        alt.Tooltip('Footprint (sq. ft.):Q', title="Footprint (sq. ft.)", format=".3f")
    ]
)

scatter = scatter.add_params(selection).transform_filter(
    alt.expr.if_(selection == "All Brands", True, alt.datum.Manufacturer == selection)
)
```

Step 4: Adding a Trend Line

- To highlight the negative correlation, we overlay a linear regression trend line.
- This visually confirms that as fuel economy (MPG) increases, CO_2 emissions decrease.

In [52]:
```
trend_line = alt.Chart(manufacturer_epa_df).transform_regression('Real-World MPG', 'Real-World CO2 (g/mi)', method='linear')

trend_line = trend_line.mark_line(color='black', opacity=0.8).encode(
    x='Real-World MPG:Q',
    y='Real-World CO2 (g/mi):Q'
)
```

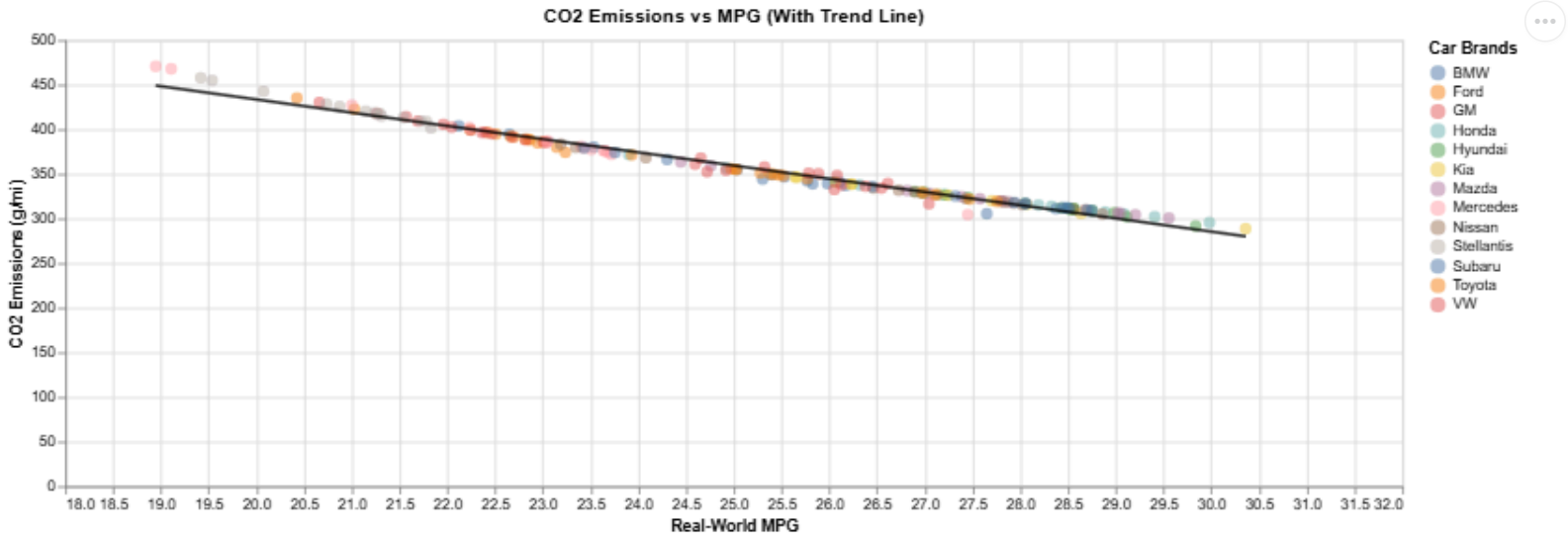
Step 5: Adding Zoom and Finalizing the Chart

- Users can zoom in and out dynamically to focus on specific regions of the chart.
- The final chart integrates dropdown selection, tooltips, a trend line, and zooming for full interactivity.

In [54]:
```
zoom_selection = alt.selection_interval(bind='scales')

multi_layer_chart_interactive = (scatter + trend_line).properties(
```

```
    title="CO2 Emissions vs MPG (With Trend Line)",
    height=300,
    width=900
)

multi_layer_chart_interactive = multi_layer_chart_interactive.add_params(zoom_selection)

multi_layer_chart_interactive
```

Out[54]:



Select Brand: All Brands

To further validate the Trendline, we can calculate these Additional Statistical Measures:

In [56]:

```
manufacturer_epa_df = manufacturer_epa_df.dropna(subset=['Real-World MPG', 'Real-World CO2 (g/mi)'])

slope, intercept, r_value, p_value, std_err = linregress(manufacturer_epa_df['Real-World MPG'], manufacturer_epa_df['Real-World CO2 (g/mi)'])

regression_results = pd.DataFrame({
    "Metric": ["Slope", "Intercept", "R-value", "R-squared", "P-value", "Std Error"],
    "Value": [slope, intercept, r_value, r_value**2, p_value, std_err]
})

regression_results
```

Out[56]:

	Metric	Value
0	Slope	-1.481132e+01
1	Intercept	7.297596e+02
2	R-value	-9.914404e-01
3	R-squared	9.829540e-01
4	P-value	4.201752e-161
5	Std Error	1.453792e-01

Conclusion:

- There is a strong negative correlation between MPG and CO_2 emissions.
- Vehicles with higher fuel economy (MPG) produce significantly lower CO_2 emissions, supporting the push for more fuel-efficient and hybrid vehicles.
- The interactive chart allows users to filter by car brand, explore detailed vehicle attributes, and zoom for deeper analysis.

Visualization #4: Distribution of MPG Across Vehicle Weights

- This box plot visualizes how vehicle weight affects fuel economy (MPG).
- The distribution highlights variations in MPG across different weight categories.
- Outliers suggest that some heavier vehicles achieve higher-than-expected MPG, possibly due to hybrid or electric powertrains.

Step 1: Adding a Dropdown for Model Year

To allow users to filter vehicles by Model Year, we create a dropdown selection.
This enables users to view data for a Model Year or choose `"All Years"` to see the complete dataset.

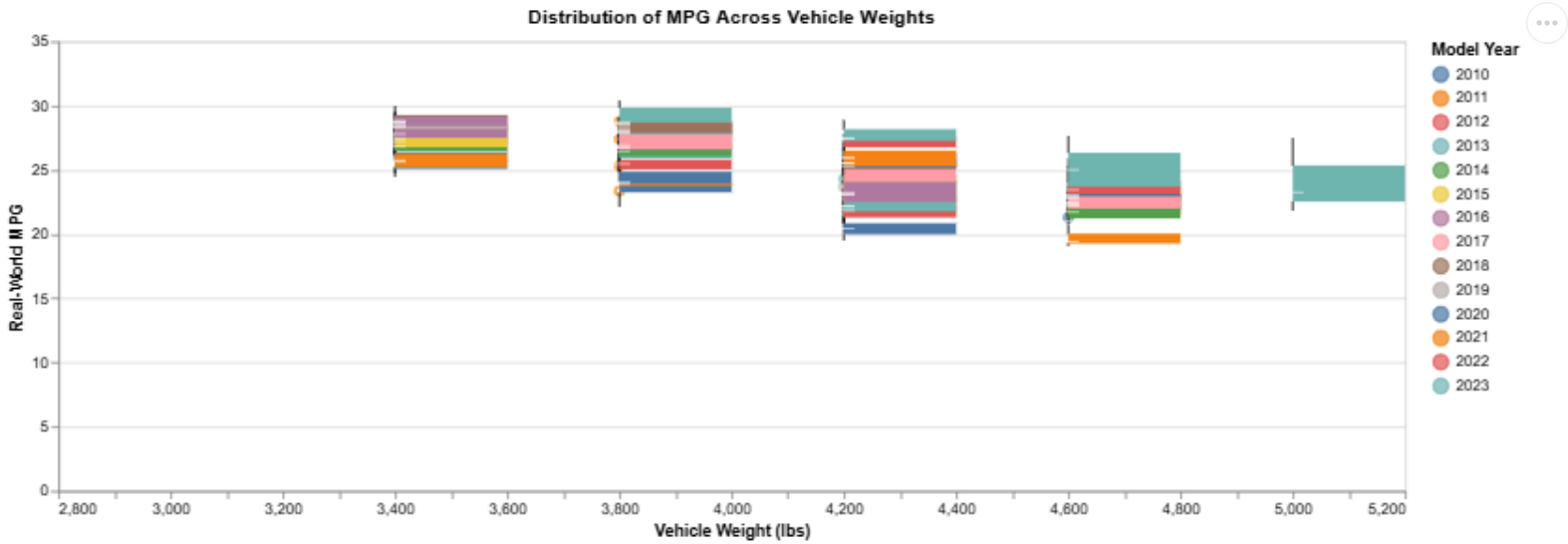
In [60]:
```
model_years = ["All Years"] + sorted(manufacturer_epa_df["Model Year"].unique().tolist())
dropdown = alt.binding_select(options=model_years, name="Select Model Year: ")
year_selection = alt.param(name="Year", bind=dropdown, value="All Years")
```

Step 2: Creating the Box Plot

Now, we create a box plot to visualize the distribution of Real-World MPG across different vehicle weight categories.

In [62]:
```
box_plot = alt.Chart(manufacturer_epa_df).mark_boxplot().encode(
    x=alt.X('Weight (lbs):Q', title="Vehicle Weight (lbs)", bin=alt.Bin(step=400)),
    y=alt.Y('Real-World MPG:Q', title="Real-World MPG"),
    color=alt.Color('Model Year:N', title="Model Year", legend=alt.Legend(title="Model Year"))
)

box_plot = box_plot.add_params(year_selection).transform_filter((alt.datum["Model Year"] == year_selection) | (year_selection == "All Years"))

box_plot = box_plot.properties(title="Distribution of MPG Across Vehicle Weights", height=300, width=900)

box_plot
```

Out[62]:



Select Model Year: All Years

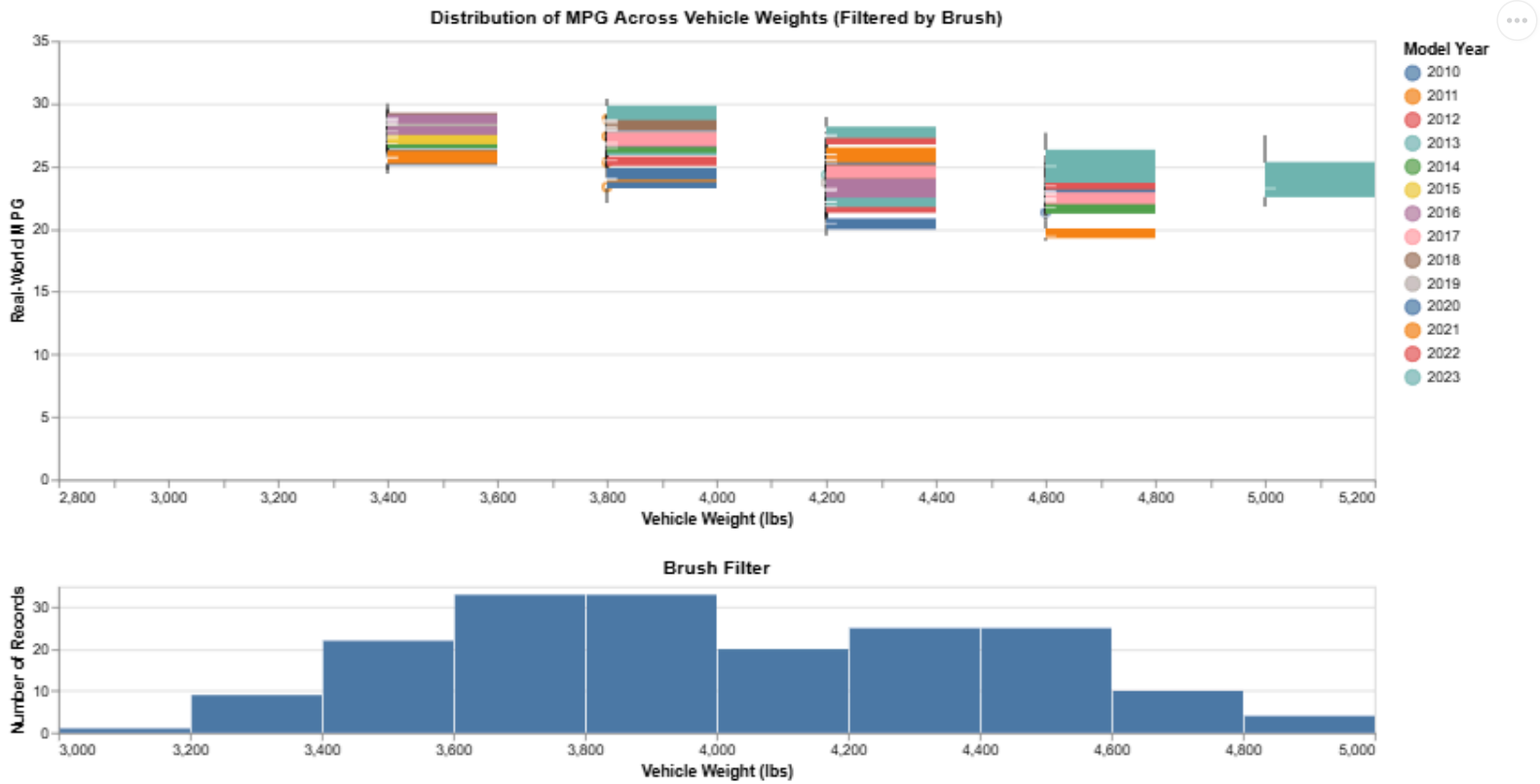
Step 3: Creating a Bar Chart for Weight Range

- To allow users to visually select a range of vehicle weights, we create a bar chart that bins weight into 200-lb increments.
- We then add the `brush` parameter for interactive filtering and set the chart's size and title.

In [64]:

```
brush = alt.selection_interval(encodings=['x'])
base = alt.Chart(manufacturer_epa_df).add_params(year_selection).transform_filter((alt.datum["Model Year"] == year_selection) | (year_selection == "All Years"))

brush_chart = base.mark_bar().encode(
    x=alt.X('Weight (lbs):Q', title="Vehicle Weight (lbs)", bin=alt.Bin(step=200)),
    y=alt.Y('count()', title="Number of Records")
)

brush_chart = brush_chart.add_params(brush)
brush_chart = brush_chart.properties(height=100,width=900,title="Brush Filter")

box_plot = base.mark_boxplot().encode(
    x=alt.X('Weight (lbs):Q', title="Vehicle Weight (lbs)", bin=alt.Bin(step=400)),
    y=alt.Y('Real-World MPG:Q', title="Real-World MPG"),
    color=alt.Color('Model Year:N', title="Model Year")
)

box_plot = box_plot.transform_filter(brush)
box_plot = box_plot.properties(height=300, width=900, title="Distribution of MPG Across Vehicle Weights (Filtered by Brush)")

final_chart = alt.vconcat(box_plot, brush_chart)
final_chart
```

Out[64]:



Select Model Year: All Years

To further validate the Box and Violin Plot, we can calculate these Additional Statistical Measures:

In [66]:

```
summary_stats = manufacturer_epa_df.groupby("Model Year")["Real-World MPG"].describe(percentiles=[0.25, 0.5, 0.75]).reset_index()

summary_stats = summary_stats.rename(columns={
    "max": "Max of Average MPG",
    "75%": "Q3 of Average MPG",
    "50%": "Median of Average MPG",
    "25%": "Q1 of Average MPG",
    "min": "Min of Average MPG"
})

summary_stats
```

Out[66]:

	Model Year	count	mean	std	Min of Average MPG	Q1 of Average MPG	Median of Average MPG	Q3 of Average MPG	Max of Average MPG
0	2010	13.0	23.252292	2.648653	18.94624	21.26286	23.43180	24.92316	27.00315
1	2011	13.0	23.108409	2.420629	19.10806	21.02658	23.75675	24.75829	26.89448
2	2012	13.0	24.364857	2.434075	20.07502	22.66097	25.03342	26.23225	28.04818
3	2013	13.0	25.133734	2.580860	20.87185	22.23815	25.88618	27.19169	28.98618
4	2014	13.0	25.360903	2.368688	20.73541	23.05124	26.08121	26.97831	29.01930
5	2015	13.0	25.784959	2.522853	21.77917	23.39981	26.06393	28.01994	29.20490
6	2016	13.0	25.845334	2.623183	21.54052	23.64929	26.23074	28.06045	29.55620
7	2017	13.0	25.912703	2.719810	21.14576	23.04015	26.37891	28.50861	29.40652
8	2018	13.0	25.967881	2.735232	21.72275	23.50904	25.98177	28.55494	29.98078
9	2019	13.0	25.896834	2.611461	21.23923	23.67964	26.16839	28.06764	28.88833
10	2020	13.0	25.976643	2.590614	21.29298	23.42644	26.98401	27.93080	29.08240
11	2021	13.0	25.960418	2.825803	21.25992	23.63750	27.10319	28.54817	28.75033
12	2022	13.0	26.010073	2.680600	21.31014	23.71639	27.04424	27.93890	29.11395
13	2023	13.0	26.951973	2.738880	21.82984	27.04400	27.57798	28.37400	30.35988

Conclusion:

- The data suggests a weak positive correlation between weight and MPG, meaning that heavier vehicles do not always have worse fuel economy.
- Modern efficiency optimizations, such as hybrid systems, turbocharging, and aerodynamics, may contribute to maintaining MPG despite increasing weight.
- The presence of outliers indicates that some heavier vehicles achieve better fuel efficiency than expected, possibly due to advancements in powertrain technology.

Conclusion: How the Visualizations Relate to Each Other

The visualizations shown throughout this analysis clearly outline key trends from the EPA automotive dataset. The line charts first highlighted a steady increase in fuel economy (MPG) over the years, mostly driven by better fuel-saving technologies and the popularity of electric cars. Next, scatter plots provided deeper insight, showing the clear connection between higher horsepower and reduced fuel economy, as well as confirming how MPG strongly links to lower CO_2 emissions.

Finally, the box plots gave another layer of detail, showing how vehicle weight categories affect MPG. Surprisingly, heavier cars sometimes achieve higher MPG than expected—likely due to new technologies in hybrid engines and other efficiency improvements. Taken together, all of these charts clearly show how different car features and tech advancements interact, driving today's trends toward more sustainable automotive performance.

In [ ]: